

Database Administration Project Report

PathEats

Generation: 11

Department: Computer Science

Specialization: Data Science

Year 2

Term 3

Team: MHOB

Course: Database Administration

Lecturer: Mr. Ang Mengchhuong

Team Members

Leng Layhok

Kong Leak Smey

Keo Seavpav

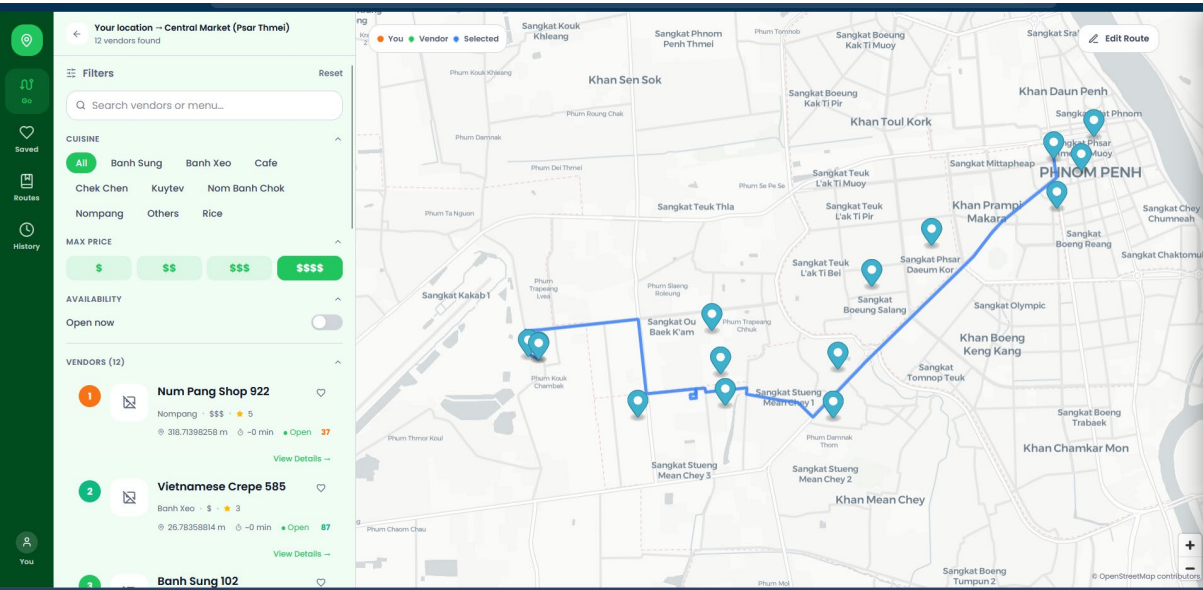
Date: 15th July 2026

Table of Contents

Table of Contents.....	2
1. Introduction	3
1.1 System Overview.....	3
1.2 Technology Stack	3
2. Team and Member Responsibilities.....	4
3. System Architecture.....	4
4. ERD and Relational Model	4
5. User Privileges and Access Control	6
5.1 Built-in Roles and Table Privileges	7
6. Role and Capability Separation	7
7. Query Optimization.....	8
7.1 Connection Pooling.....	9
7.2 Indexing Strategy	9
7.3 PostGIS Filtering and Pagination.....	10
8. Backup and Recovery.....	10
8.1 Backup.....	10
8.1.1 Manual Backup.....	10
8.1.2 Scheduled Backup	11
8.1.3 Backup Type Interfaces	11
8.2 Recovery.....	12
8.2.1 PostgreSQL Dump Recovery	13
8.2.2 Row-Level CSV Recovery.....	13
8.2.3 Recovery Controls and Limitations	13
8.2.4 Recovery Interface	14
9. Additional Database Administration Features.....	14
9.1 Database Query Tools	14
9.2 Table Health Monitoring.....	15
9.3 System Activity Monitoring.....	16
10. Conclusion.....	16
11. References	16

1. Introduction

PathEats is a location-based food discovery and vendor management platform. It helps consumers find nearby food stalls and restaurants while giving vendors tools to manage their stalls and reusable menu catalogue. The system is built as a full-stack web application with a Node.js/Express backend, a React frontend, and a Supabase-hosted PostgreSQL database extended with PostGIS for geospatial queries.



1.1 System Overview

- Geospatial search and routing are powered by the PostGIS extension on top of PostgreSQL.
- Five application roles are supported: Consumer, Vendor, Business Assistance, Global Admin, and Developer Admin.
- Supabase Storage holds uploaded images (place, menu, and profile photos) separately from relational data, with the database keeping only object references.

1.2 Technology Stack

Layer	Technology
Backend	Node.js (Express 4), JWT authentication, bcryptjs, Multer
Database	PostgreSQL (Supabase) with the PostGIS extension
Frontend	React 18, React Router 7, MapLibre GL, Tailwind CSS, Axios, OSRM, OpenFreeMap
Tooling	Vite, ESLint, Vitest, Swagger

2. Team and Member Responsibilities

The project was developed by Team MHOB, with responsibilities divided by application area so each member owned an end-to-end vertical slice of the system:

Member	Responsibility
Keo Seavpav	Vendor side and Business Assistance side (stall management, menu catalogue, onboarding support)
Leng Layhok	Developer side and Global Admin side (role/privilege management, backup and recovery, database tooling)
Kong Leak Smey	Consumer side (place discovery, routing, reviews, bookmarks)

3. System Architecture

The backend follows a layered architecture that keeps HTTP routing, business rules, and SQL access separate:

- Routes (routes/) — thin route definitions per domain: auth, users, places, vendor, admin.
- Middlewares (middlewares/) — authentication, RBAC/ownership guards, privilege checks, and rate limiting.
- Services (services/) — business logic, transactions, and validation (AuthService, PlaceService, VendorService, AdminService, backup/recovery services).
- Repositories (repositories/) — parameterized SQL access per entity (UserRepository, PlaceRepository, VendorRepository, TokenRepository, OtpRepository).

This separation keeps routes free of SQL, centralizes transaction handling in services, and makes ownership and privilege rules easy to audit and test.

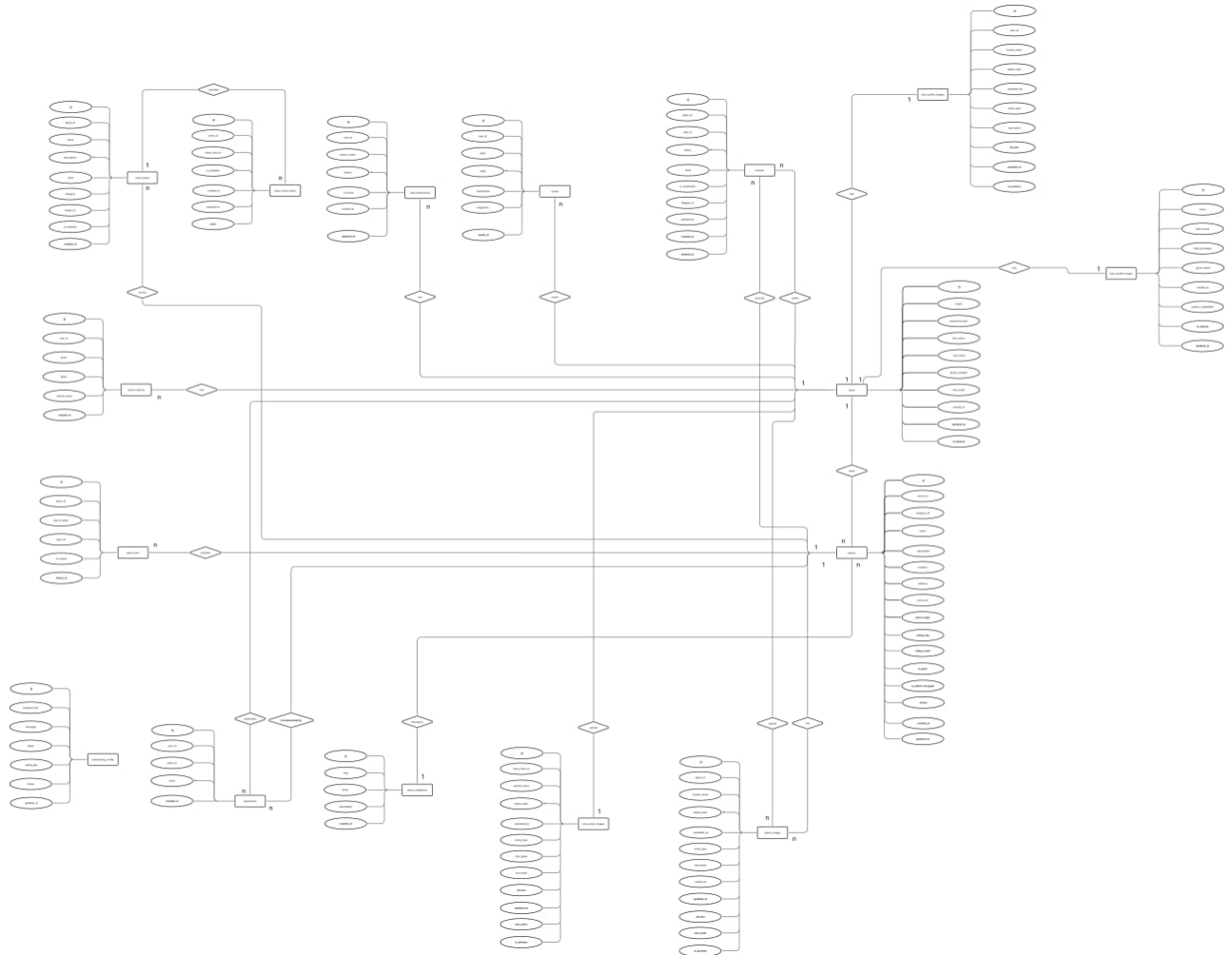
4. ERD and Relational Model

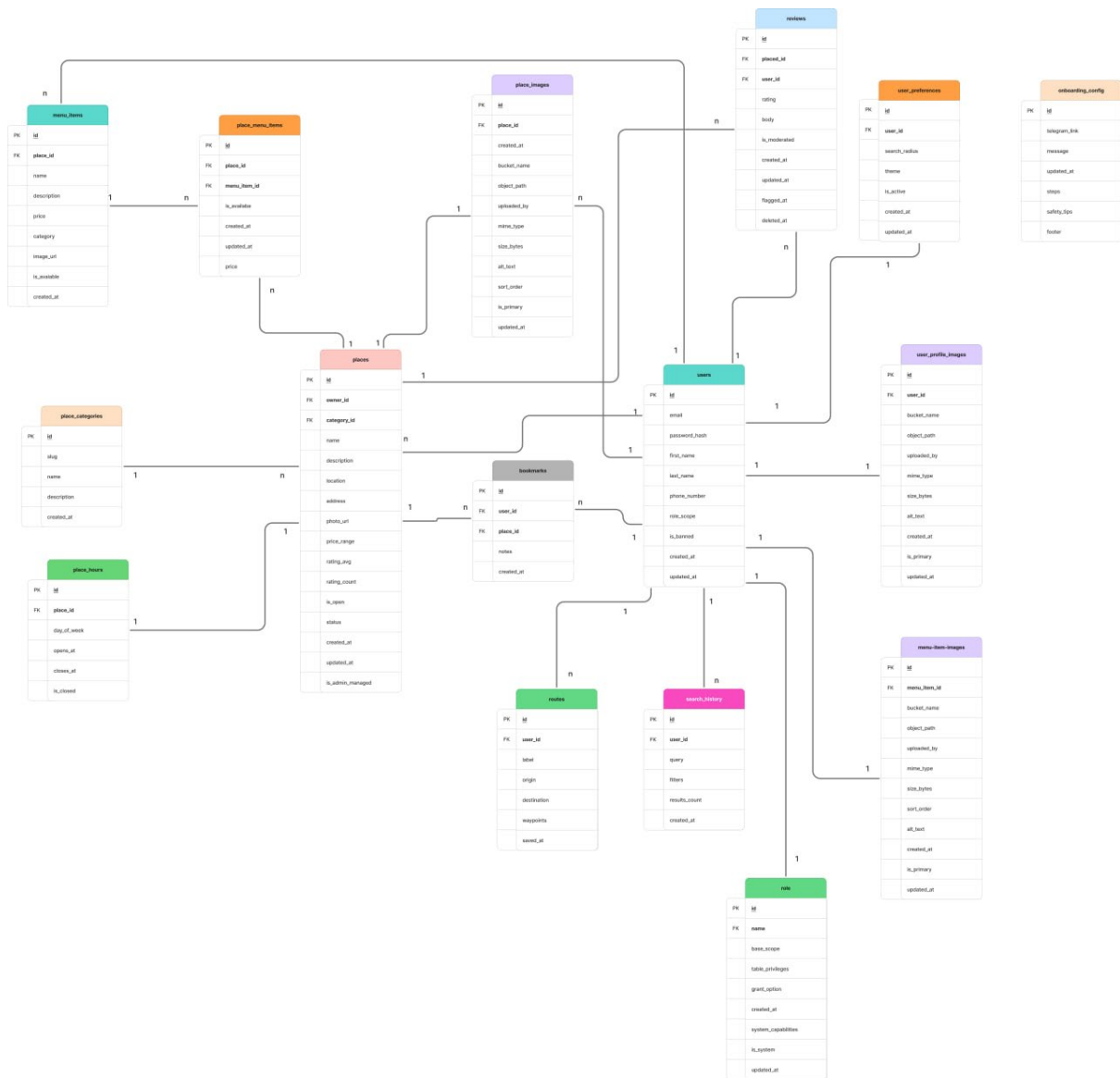
The relational model centers on users and their roles, the places (stalls/restaurants) a vendor owns, and a shared menu catalogue that can be linked to multiple stalls. Key entities include:

- Users: accounts with role_scope and a role_id foreign key into the role table.
- Role: application roles with a per-table privilege matrix (table_privileges) and system-level capabilities (system_capabilities).
- Places: vendor stalls/restaurants with a PostGIS location column, owner_id, category, and status.
- place_hours, place_images: operating hours and photos linked to a place.
- menu_items: a vendor's reusable menu catalogue, independent of any single stall.
- place_menu_items: the join table linking a catalogue item to a specific place, carrying stall-specific price and availability.
- reviews, routes, bookmarks, search_history: consumer-generated data tied to users and places.

- `backup_profiles`, `scheduled_backups`, `recovery_operations`, `database_activity_log`: operational tables supporting backup, recovery, and auditing.

Splitting `menu_items` (the catalogue) from `place_menu_items` (the per-stall link) allows a vendor to reuse one menu identity across several stalls while keeping stall-specific price and availability independent. Shared catalogue fields such as name, description, and category update across linked stalls, while price and availability affect only the selected stall.





Link to full diagram of ERD and RM: [Open diagram in Figma](#)

5. User Privileges and Access Control

Access control is enforced at the application layer rather than through native PostgreSQL role impersonation. Every authenticated request is resolved to a role row that carries a JSON privilege matrix and a list of system capabilities:

```
const token = authHeader.split(" ")[1];
const decoded = jwt.verify(token, JWT_SECRET);
const { rows } = await db.query(
  `SELECT u.id, u.email, u.role_scope, u.role_id::text, u.is_banned,
    r.name AS role_name, r.base_scope, r.table_privileges,
    r.system_capabilities, r.grant_option, r.is_system
  FROM users u
  JOIN "role" r ON r.id = u.role_id
  WHERE u.id = $1
  LIMIT 1`,
  [decoded.sub]
);
```

The decoded JWT only carries the user's id; the full privilege matrix is loaded fresh from the database on every request instead of being embedded in the token, so a privilege change takes effect immediately without waiting for token expiry.

5.1 Built-in Roles and Table Privileges

Role	Representative table privileges
GLOBAL_ADMIN	Full SELECT / INSERT / UPDATE / DELETE across all managed tables (users, places, menu, role, backup, audit, etc.)
BUSINESS_ASSISTANCE	Manage users, places, menu items, reviews (moderation); read-only on audit_log.
DEVELOPER_ADMIN	No general table CRUD privileges by default; access to backup, recovery, query, and maintenance tools is granted through system capabilities.
VENDOR	Full control over own places, hours, images, and menu items; read-only on reviews
CONSUMER	Read places/menu; full control over own reviews, routes, bookmarks, and search history

6. Role and Capability Separation

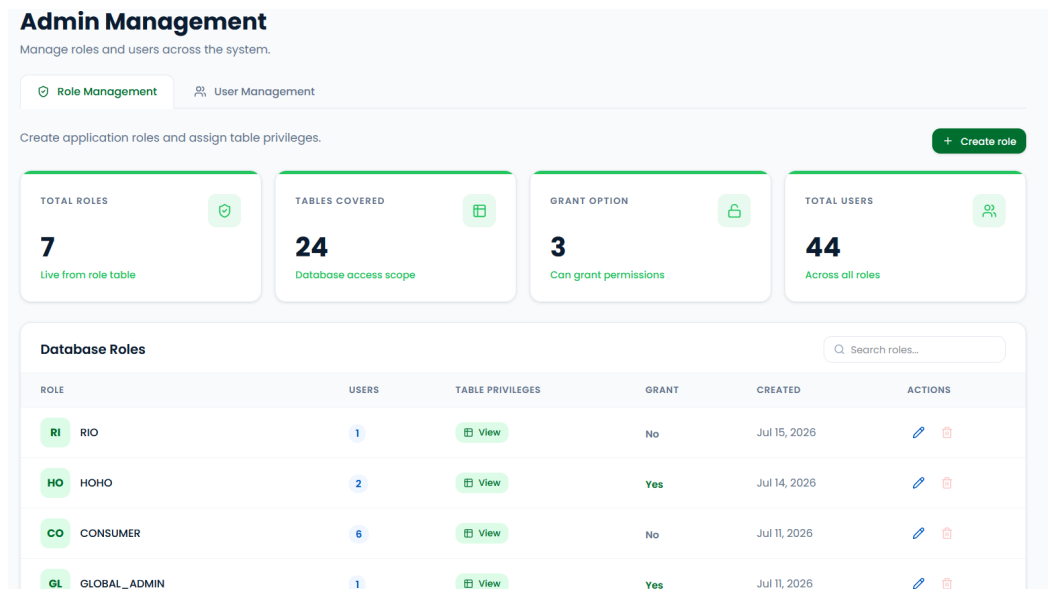
PathEats deliberately separates two kinds of permission so that operational access (backups, recovery, maintenance) can never be granted just by handing out table privileges:

- Table privileges — CRUD-style access (SELECT/INSERT/UPDATE/DELETE) scoped to a specific application table, defined in a privilege registry.
- System capabilities — BACKUP, RECOVERY, QUERY, and MAINTENANCE, which gate operational tooling independent of table access.

```
// Developer operations
router.use(restrictToRoles("GLOBAL_ADMIN", "DEVELOPER_ADMIN"));
router.use("/backups", requireSystemCapability("BACKUP"));
router.use("/recovery", requireSystemCapability("RECOVERY"));
```

Role access and system capabilities are checked separately, so a role can be given broad table access without automatically being able to run backups or restores, and vice versa.

Built-in roles (CONSUMER, VENDOR, GLOBAL_ADMIN, DEVELOPER_ADMIN, BUSINESS_ASSISTANCE) cannot be renamed or deleted, and Global Admin retains a minimum set of privileges to prevent an administrator from accidentally locking themselves out. Custom roles created through the admin role editor expose only a name, a per-table CRUD matrix, and a single role-wide Grant Option — administrators cannot assign raw database-level CREATE/ALTER/DROP privileges or edit sensitive implementation tables such as refresh tokens, OTPs, or sessions.



7. Query Optimization

PathEats optimizes its common database access paths using four complementary techniques:

- PostgreSQL connection pooling, so requests reuse existing connections.
- B-tree and PostGIS GiST indexes on the columns most frequently filtered or searched.
- Early filtering and pagination, so PostgreSQL narrows result sets before rows cross the network.
- Parameterized queries and targeted joins, avoiding SQL injection risk and unnecessary joined data.

7.1 Connection Pooling

```
const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production'
    ? { rejectUnauthorized: true }
    : { rejectUnauthorized: false },
  max: 20,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 10000,
  keepAlive: true,
});
```

Requests reuse a shared pool of up to 20 connections instead of opening a new database connection for every query, reducing connection-setup overhead under load.

7.2 Indexing Strategy

Indexes are grouped by the access pattern they support:

Index	Supports
GiST on places(location)	Nearby and route-based place searches
places(owner_id)	Vendor-owned place filtering
menu_items(owner_id)	Vendor catalogue filtering
place_menu_items(place_id, is_available)	Available-menu lookup for a given place
Unique partial index on reviews(place_id, user_id) WHERE deleted_at IS NULL	Prevents a consumer from holding more than one active review per place
users(email), users(role_scope)	Login lookup and role-based filtering
audit_log / database_activity_log timestamps	Recent-activity queries in admin dashboards
Preferred clustering index: places(owner_id, created_at DESC)	Groups vendor-owned places in newest-first order when CLUSTER is run during maintenance

```
CREATE INDEX IF NOT EXISTS idx_places_location ON places USING GIST (location);
CREATE INDEX IF NOT EXISTS idx_places_owner ON places(owner_id);
CREATE INDEX IF NOT EXISTS idx_places_owner_created_at
  ON places(owner_id, created_at DESC);
ALTER TABLE places CLUSTER ON idx_places_owner_created_at;
CREATE INDEX IF NOT EXISTS idx_menu_items_owner ON menu_items(owner_id);
CREATE INDEX IF NOT EXISTS idx_place_menu_items_available
  ON place_menu_items(place_id, is_available);

CREATE UNIQUE INDEX IF NOT EXISTS idx_reviews_unique_active
  ON reviews (place_id, user_id)
  WHERE deleted_at IS NULL;
```

PostGIS provides GiST indexes for spatial data and functions such as `ST_Distance` for exact distance calculations. The indexing script also records `idx_places_owner_created_at` as the preferred clustering index for vendor listings; physical clustering is a maintenance operation and is not automatically maintained after later writes.

7.3 PostGIS Filtering and Pagination

Nearby-search and routing endpoints build a PostGIS geography filter alongside optional category, price, and name conditions, so PostgreSQL — not the application layer — performs the geospatial distance calculation and filtering:

```
const conditions = [
  `p.status = 'active'`,
  `p.is_open = TRUE`,
  `p.deleted_at IS NULL`,
  `ST_Distance(p.location::geography, ST_GeomFromGeoJSON($1)::geography) <= $2`,
  `($3::text IS NULL OR pc.name = $3)`,
  `($4::int IS NULL OR p.price_range <= $4)`,
  `($5::text IS NULL OR p.name ILIKE $5 OR mi_search.names IS NOT NULL)`,
  `($6::boolean = FALSE OR ${OPEN_NOW_SQL})`,
];
```

8. Backup and Recovery

PathEats protects PostgreSQL application data through authorized manual and scheduled backups. Access is limited to Global Admin and Developer Admin users who possess the required BACKUP or RECOVERY system capability.

8.1 Backup

The common process is profile configuration, validation, artifact generation, download or storage, history update, and temporary-file cleanup.

8.1.1 Manual Backup

- **Entire Database:** Creates a PostgreSQL custom-format `.dump` of the PathEats public schema. It excludes PostgreSQL system schemas, Supabase-managed schemas, and `spatial_ref_sys`.
- **Specific Tables:** Creates a custom-format `.dump` containing only the selected and validated public tables.
- **Specific Rows:** Exports selected records from one table as CSV. The optional WHERE condition is validated; without a condition, all rows in that table are exported.
- **Execution:** The artifact is generated on demand, streamed to the administrator, and removed from temporary storage after the response finishes.

PostgreSQL dump command

```
await runPostgresTool("pg_dump", [
  "--format=custom",
  "--no-owner",
  "--no-privileges",
  "--no-comments",
  "--file",
  dumpPath,
  ...scopeArgs,
]);
```

8.1.2 Scheduled Backup

- **Backup types:** Scheduled profiles support the same Entire Database, Specific Tables, and Specific Rows types.
- **Execution:** The scheduler checks every 60 seconds and claims up to five due profiles using FOR UPDATE SKIP LOCKED to reduce duplicate execution.
- **Storage:** Completed artifacts are copied to the configured backend backup directory, while execution metadata is recorded in scheduled_backups.
- **History:** The profile status, last and next run times, file size, run count, and any error message are updated after execution.

8.1.3 Backup Type Interfaces

Entire Database

Create Backup Profile

×

Profile Name

e.g. Daily_Full_Main_DB

Backup Method

Entire Database

Specific Tables

Specific Rows

Target Schema

public

Creates a PostgreSQL custom backup of the current database public schema. It can be restored with the PostgreSQL Dump recovery option.

Schedule (optional)

0 = none

Hours

Set to 0 or leave empty for no schedule.

Cancel

Create & Download

Specific Tables

Create Backup Profile

×

Profile Name

e.g. Daily_Full_Main_DB

Backup Method

Select Tables

☐ audit_log	☐ backup_profiles	☐ bookmarks
☐ database_activity_log	☐ menu_item_images	☐ menu_items
☐ onboarding_config	☐ otps	☐ place_categories
☐ place_hours	☐ place_images	☐ place_menu_items
☐ places	☐ query_presets	☐ recovery_operations
☐ refresh_tokens	☐ reviews	☐ role

Schedule (optional)

0 = none Hours

Set to 0 or leave empty for no schedule.

Specific Rows

Create Backup Profile

×

Profile Name

e.g. Daily_Full_Main_DB

Backup Method

Select Table

— Select —

WHERE Condition (optional)

e.g. WHERE created_at > NOW() - INTERVAL '30 days'

Downloads a CSV containing only matching rows. Restoring it updates matching primary keys and preserves unrelated data.

Schedule (optional)

0 = none Hours

Set to 0 or leave empty for no schedule.

8.2 Recovery

Recovery requires authorization, an uploaded file, the required confirmation phrase, and format-specific validation before any database changes are attempted.

8.2.1 PostgreSQL Dump Recovery

- **Validation:** The service verifies the custom-dump signature, runs `pg_restore --list`, and filters the restoration list to supported objects in the public schema.
- **Full dump:** A temporary safety dump is created before the public schema is restored with `--single-transaction` and `--exit-on-error`. A failed restore does not commit partial changes.
- **Partial-table dump:** Recognized tables are restored individually. Each table uses its own transaction, but a group of several tables is not atomic as one complete operation.

PostgreSQL restore command

```
await runPostgresTool("pg_restore", [
  "--clean",
  "--if-exists",
  "--exit-on-error",
  "--single-transaction",
  "--no-owner",
  "--no-privileges",
  tableArg,
  "--use-list",
  restoreList.listPath,
  "--dbname",
  postgresToolConfig().databaseName,
  dumpFilePath,
]);
```

8.2.2 Row-Level CSV Recovery

- **Validation:** The target table and CSV headers are checked against the current public schema, and unsupported columns are rejected.
- **Restore:** Rows are processed with parameterized statements in one database transaction. Complete primary keys enable upsert behavior; otherwise, safe conflicts are ignored.
- **Failure handling:** If any CSV row fails, the transaction is rolled back, and no CSV changes are committed.

8.2.3 Recovery Controls and Limitations

- **Concurrency:** A process-local lock prevents overlapping recovery operations in one backend instance. It does not coordinate multiple backend instances.
- **Audit and cleanup:** Results are recorded in recovery history and the administrative audit log and uploaded, or temporary files are removed after processing.
- **Scope limitation:** Database backups include relational records and image metadata, but not the actual objects stored in Supabase Storage. Scheduled artifacts are currently stored on the backend filesystem rather than off-site storage.

8.2.4 Recovery Interface

Recovery file selection

Initiate Recovery

Recovery Type

PostgreSQL Dump
Row Level CSV

test-backup.dump Remove

Cancel
Review & Confirm

Destructive-action confirmation

Confirm Recovery

This action cannot be undone
You are about to execute PostgreSQL Dump from test-backup.dump. Existing data may be overwritten or modified.

Type RESTORE POSTGRES DUMP to confirm
A verified full database dump restores the public schema in one transaction. A failed restore commits no database changes.
RESTORE POSTGRES DUMP

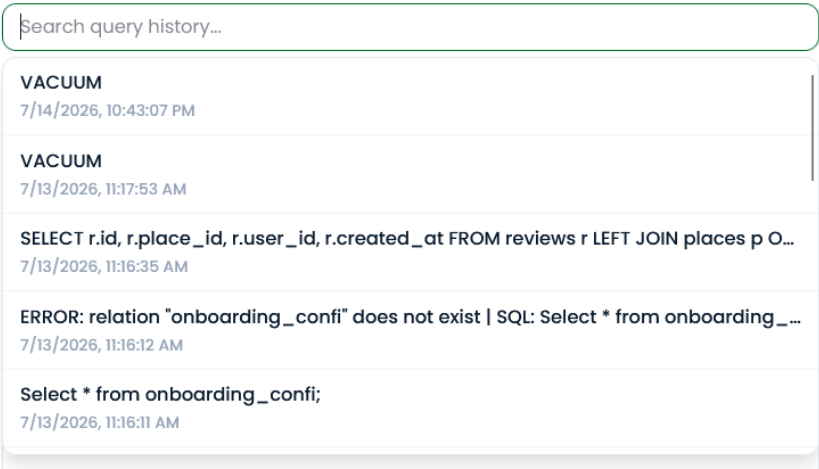
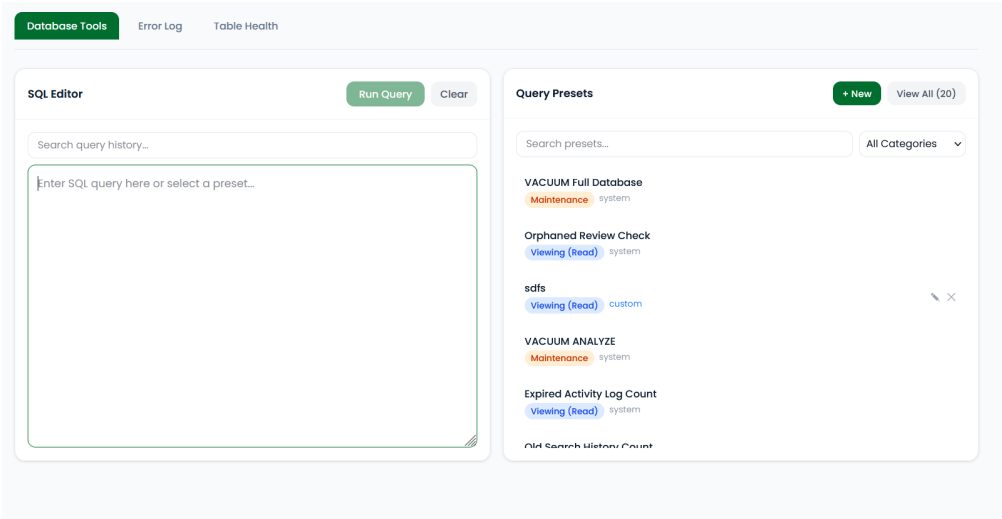
Back
Confirm & Execute Recovery

9. Additional Database Administration Features

PathEats includes supporting tools for controlled database investigation, maintenance, and operational monitoring.

9.1 Database Query Tools

The developer and global administrator portals provide a controlled SQL interface, reusable query presets, and query history. Access is restricted by role and the QUERY system capability.



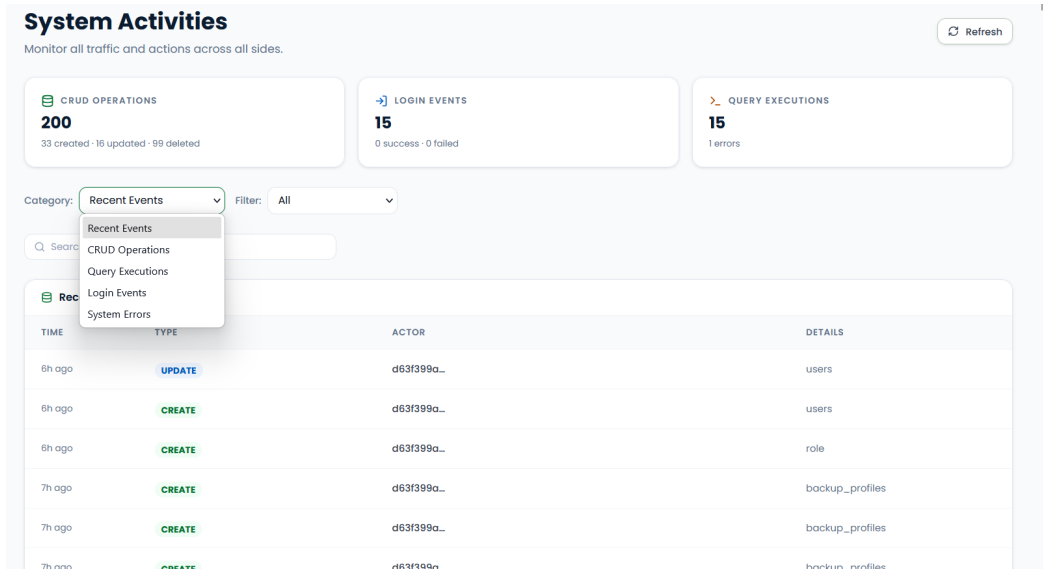
9.2 Table Health Monitoring

The Table Health view summarizes table size, dead tuples, and maintenance status so administrators can identify tables that may require analysis or vacuuming.

Database Tools Error Log Table Health			
Table Health Monitor table size, dead tuples, and maintenance status from PostgreSQL statistics. Refresh			
TABLES 61	GOOD 57	WARNING 2	CRITICAL 2
TABLE	DEAD TUPLES	SIZE	HEALTH
query_presets	27	88 kB	critical
refresh_tokens	20	240 kB	warning
menu_items	6	312 kB	good
objects	3	96 kB	critical
backup_profiles	2	80 kB	warning
places	1	808 kB	good
users	1	136 kB	good
oauth_clients	0	24 kB	good
webauthn_credentials	0	32 kB	good
...	...	...	...

9.3 System Activity Monitoring

The System Activities view records CRUD operations, login events, query executions, and errors to support accountability and maintenance auditing.



10. Conclusion

PathEats demonstrates a practical application of database administration principles on top of a managed Supabase/PostgreSQL instance: a clear relational model with a reusable menu catalogue, application-enforced role-based access control with a granular privilege registry, geospatial querying with PostGIS, deliberate indexing and connection pooling for performance, and a transaction-safe backup and recovery workflow. Together these choices keep the system performant, auditable, and safe to operate as the team's course project for Database Administration.

11. References

- Postgis: <https://www.postgis.net/workshops/postgis-intro/>
- Supabase: <https://supabase.com/docs>
- Express.js: <https://expressjs.com/>
- MapLibre: <https://maplibre.org/>
- Code repository: <https://github.com/Layhok14/PathEats.git>